

PRE-CLASS READING 2 OF 5

The loop itself — ReAct: reasoning and acting, interleaved

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. (2022). "ReAct: Synergizing Reasoning and Acting in Language Models." *arXiv:2210.03629* (v1 6 Oct 2022; v3 10 Mar 2023); published at ICLR 2023. This is the paper the agent loop in your Capsule 2 notebook is built from.

ASSIGNED READING

Read the abstract, Section 1 (introduction) and Section 2 (the method), and — this is the part that actually teaches — **study one full worked trajectory** in the figures, line by line. The experimental sections can be skimmed. (≈ 25 min)

» source ~25 min • this summary + reasoning notes ~18 min

Note on currency. This is a 2022 paper and the models it tested (PaLM-540B, early GPT-3) are long superseded. Read it for the **mechanism**, not the scores. Every tool-calling loop you will use in 2026 — including the one in Capsule 2 — is a ReAct loop with better plumbing; the paper is worth reading precisely because modern frameworks hide the loop behind an API and you stop being able to debug what you cannot see.

KEY TAKEAWAY

Before ReAct, two things were studied separately: models that **reason** in text (chain-of-thought — good at planning, but sealed inside their own heads, so a wrong premise never gets corrected) and models that **act** (emit an action, get a result — grounded, but with no visible plan, so they wander). ReAct's move is almost embarrassingly simple: **interleave them in one token stream**. The model writes a thought, takes an action, reads the observation the world returns, and then writes the next thought conditioned on what actually happened. Reasoning tells the actions where to go; observations keep the reasoning honest. That alternation — **Thought → Action → Observation, repeat** — is the agent loop, and everything in Week 4 is a variation on it.

WHY IT MATTERS FOR THIS COURSE

Reading 1 describes agents from the outside — when to build one. This reading is the inside: what the loop *is*, as text. It matters for Capsule 2 because you will build this loop and then break it deliberately, and you cannot debug a trajectory you cannot read. It also gives you the cleanest possible statement of a Week 3 idea that now pays off: an agent is still just next-token prediction. The thoughts are tokens. The actions are tokens. The observations are tokens pasted back into the context. Which means an agent's bill, its context-window pressure and its failure modes are all the same objects you priced in Week 3 — just multiplied by the number of turns.

Feeds Class 4: C1 (the control loop • ReAct vs planner-executor) • C2 (the build • reading traces • failure modes) • C3 (why turns multiply the token bill).

MAIN POINTS

Quoted phrases and reported figures are the paper's. Anything after Our reading is this brief's interpretation — in particular the worked trajectory below and the cost arithmetic, neither of which appears in a 2022 paper.

The mechanism, in one worked trajectory

Our example, not the paper's — the shape is theirs. Task: "Which Singapore bank reported the higher net profit in FY2024, and by how much?" A ReAct trajectory looks like this — every line is text the model either wrote or was given:

- **Thought 1.** "I need two numbers: DBS FY2024 net profit and OCBC FY2024 net profit. Search for the first."
- **Action 1.** ``search("DBS FY2024 net profit")``
- **Observation 1.** (returned by the tool, pasted into the context) "DBS full-year 2024 net profit S\$11.4bn..."
- **Thought 2.** "Got one. Now OCBC." → **Action 2.** ``search("OCBC FY2024 net profit")`` → **Observation 2.** "OCBC FY2024 net profit S\$7.59bn..."
- **Thought 3.** "Both in hand. $11.4 - 7.59 = 3.81$. Subtraction is a place I make mistakes; use the calculator rather than assert it." → **Action 3.** ``calculate("11.4 - 7.59")`` → **Observation 3.** "3.81"
- **Thought 4.** "I can answer." → **Action 4.** ``finish("DBS, by about S$3.8bn.")``

Three things to notice, because they are the whole paper. (i) The observations are *not* generated by the model — they are pasted in by your code, and that is the only point where the outside world can contradict it. (ii) The thoughts are not decoration; they are where the model decides *which* action comes next, so removing them (act-only prompting) degrades performance badly on multi-hop tasks — that ablation is the paper's. (iii) The loop is **stateless between turns** — the only thing carrying it forward is the growing text of the trajectory. **Our reading:** (i) and (iii) are how we frame the mechanism for this course; the paper demonstrates them rather than stating them this way.

Why interleaving beats either half

- **Chain-of-thought alone hallucinates and cannot recover.** Reasoning with no external input is a closed system: if the model misremembers a fact in step 2, every later step builds on it, confidently. The paper's framing is that ReAct "overcomes issues of hallucination and error propagation prevalent in chain-of-thought reasoning by interacting with a simple Wikipedia API."
- **Acting alone wanders.** Without written-down reasoning the model has no plan to hold across turns — it can look things up but not decide what to look up next, so multi-hop tasks fall apart.
- **Together they also become legible.** The paper stresses "more interpretable and trustworthy" trajectories: because the plan is written in the same stream as the actions, a human can read the run and see *where* it went wrong, not merely *that* it did. Underrated in practice — most of your Capsule 2 debugging will be trajectory-reading.
- **Best-of-both, formally.** The strongest results in the paper come from combining ReAct with chain-of-thought self-consistency — internal knowledge and external evidence, switching between them — rather than from either alone.

What the numbers said (and how to read them)

- **Knowledge tasks (HotpotQA multi-hop QA, FEVER fact verification).** ReAct beat act-only clearly and traded places with chain-of-thought; the combination beat both. The honest summary is that grounding removed a class of confident errors rather than making the model cleverer.
- **Decision tasks — the striking result.** On **ALFWorld** (text-based household tasks) ReAct improved absolute success rate by **34 percentage points** over imitation-learning and reinforcement-learning baselines; on **WebShop** (online shopping) by **10 points** — while using only **one or two in-context examples**, against baselines trained on thousands of demonstrations. That contrast, not the absolute score, is the finding: a prompt with two examples beat a trained policy.
- **Do not carry the scores forward.** They are 2022 numbers on 2022 models. The mechanism generalised; the benchmarks did not survive contact with the frontier.

Failure modes the paper is honest about — and you will meet in class

- **Search-result dependence.** When the retrieval step returns unhelpful text, the trajectory derails: the model reasons impeccably about the wrong document. Garbage observation in, confident nonsense out.
- **Repetitive loops.** The model reissues the same action, gets the same observation, writes a slightly reworded thought, and repeats — burning tokens while making no progress. This is the single most common agent failure you will see live, and the standard fixes are all outside the model: a step cap, deduplication of repeated actions, and a cost ceiling.
- **Reasoning that overrides evidence.** Occasionally the model reads a correct observation and then argues past it, because its prior is strong. Grounding is a channel, not a guarantee.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall. A clue sits under each question to give you a direction to push on; the fuller reasoning is in the section after.

- In the trajectory above, exactly one line is not written by the model. Which one — and why does that single line do all the work?

Clue. Trace where information enters the system rather than imagining the model's intentions. Ask of each line: could this text exist if the outside world did not? Then ask what the loop degenerates into if you delete that line type — you will have described chain-of-thought, and the paper's whole argument, in one sentence.

- A ReAct loop that runs eight turns costs far more than eight times a single call. Work out roughly why, then say what you would change first to bring the bill down.

Clue. Do the arithmetic the other way round. Each turn resends the entire trajectory so far, so context grows every turn and you pay for the history again and again — for T turns of roughly equal size the input cost goes like $T^2/2$, not T . Try it with a 1,000-token base prompt and 300 tokens added per turn before you decide what to cut; the answer that falls out is usually about **observation size**, not about the number of turns.

- You watch your agent issue the same search three times in a row. Name three fixes at three different layers — the prompt, the tool, and the code around the loop — and say which you would ship first.

Clue. Force yourself to put one fix at each layer before you rank them; the instinct is to write three prompt fixes. Then judge each on whether it makes the failure impossible or merely less likely, and on who pays for it on every call. Carry Reading 1's poka-yoke idea over — a loop-level counter and an action-deduplication check are constraints; a sentence saying "do not repeat yourself" is a persuasion.

- ReAct beat a policy trained on thousands of demonstrations using two in-context examples. What does that tell you about where the capability actually lives — and what would you now ask a vendor who says they “trained a custom agent” for your industry?

Clue. Separate the two things a system can contain: task-specific knowledge and general reasoning capability that gets pointed at a task. Ask which one the training bought, and what happens to each when the task shifts by 10%. This is the build-versus-buy reflex from Week 2, arriving in a new costume — and it should make you ask what the vendor’s eval set looks like before you ask what their model is.

- The paper’s honest failure list is: bad retrieval derails reasoning, loops repeat, and priors sometimes override evidence. Which of those three would hurt most in your own domain, and what would you instrument to catch it before a user does?

Clue. Pick your domain’s expensive mistake first, then work backwards to the failure mode that produces it — not the other way round. For each, ask what a log would have to record for you to notice it automatically: repeated actions are trivial to count; a reasoning step that contradicts its own observation needs a judge. That difference in detectability is itself the answer to “which hurts most”.

HOW TO THINK ABOUT THESE — read after you have had a go

These are not model answers to be memorised — they are the line of reasoning we will pick up in class. Class time is short, so they are written to stand on their own.

Q1 — The observation is the only door to reality

Thoughts and actions are both produced by the model; only observations come from outside. That asymmetry is the entire design. Delete observations and the trajectory collapses into chain-of-thought: fluent, self-consistent, and unfalsifiable. Keep them and you have created the one place where the world can say “no”. Two consequences follow, and both matter in Capsule 2. First, **the quality of an agent is bounded by the quality of its observations** — a search tool that returns three paragraphs of marketing copy will produce an agent that reasons beautifully about marketing copy. Second, observations are the **most expensive** part of the loop, because whatever a tool returns gets pasted into the context and then resent on every subsequent turn. Which means the design of what a tool returns — a number, not a webpage; a row, not a table — is simultaneously an accuracy decision and a cost decision. That is a rare and useful thing in engineering: a knob where the cheap choice is also the accurate one.

Q2 — Why turns are quadratic, not linear

Our reading — this arithmetic is not in the paper; it follows from how the API bills. A single call bills you for prompt plus completion. A ReAct loop resends everything each turn: turn 3 pays for the system prompt, the tool schemas, and the whole of turns 1 and 2, plus the new observation. With a base prompt of 1,000 tokens and roughly 300 tokens added per turn, eight turns costs on the order of $8 \times 1,000 + 300 \times (1+2+\dots+7) \approx 16,400$ input tokens — against 1,300 for a single call, so about **12×** for 8 turns, not 8×. The general shape is $\text{base} \times T + \text{delta} \times T^2/2$. Two practical consequences. **Observation size is the highest-leverage knob**, because it enters the sum and is then paid for again on every remaining turn: trimming a tool’s return from 800 tokens to 80 saves far more than removing a turn. And **caching and compaction stop being optimisations and become architecture** — prompt caching flattens the repeated prefix, compaction truncates the quadratic term. If you take one number from Week 4 into your A2 cost model, take this one: agent cost grows with the square of the conversation, and the observation is the term you control.

Q3 — Three layers, and which to ship

Prompt layer: instruct the model to check whether an action has already been tried. Cheap to write, paid on every call, unreliable — a persuasion. Tool layer: have the search tool return a marker when called twice with the same query, or make the tool signature require a distinct query. Better, because it makes the loop *notice* rather than *remember*. Code layer: a hard step cap, a token-budget ceiling, and a dedupe check that terminates or escalates when the same action recurs. Ship the code-layer fix first, and ship it before you ship the agent at all — it is the seatbelt, and its virtue is that it fails **safe and loud**: the run stops, you get a log, no one is billed for four hundred identical searches at 3 a.m. Then add the tool-layer fix, because it turns a hard stop into a recovery. The prompt fix is last and optional; it is the only one that stops working when you change model. The general principle to carry into A2: **guarantees belong in code, judgement belongs in the prompt**. Repetition is not a judgement call.

Q4 — Where the capability lives

The finding is that two examples of *format* plus a strong general model beat thousands of examples of *behaviour* plus a weak specialised one. The capability lived in the base model; what the prompt supplied was a shape for it to pour into. That inverts the intuition many organisations still budget for — that domain performance must be bought with domain training. Sometimes it must, and Week 5 will look at where data genuinely is the moat. But the default order of attack in 2026 runs: better prompt and examples → better tools and retrieval → better context management → and only then fine-tuning. So the question to a vendor selling “a custom agent trained for your industry” is not really about the model. Ask: **what does your eval set contain, who wrote it, and how does the system do on cases it has never seen?** If the domain knowledge is in retrieved documents and a good tool schema,

you can inspect it, update it next week, and swap the model next quarter. If it is baked into weights, you cannot do any of the three — and you have bought a moat that faces the wrong way.

Q5 — Detectability is the real ranking

The three failure modes differ in how cheaply you can see them. **Repetition** is free to detect — hash the action, count repeats, alarm at three. **Bad-retrieval derailment** is moderately detectable: log every observation alongside the thought that consumed it, and sample-review, or score relevance with a cheap model judge. **Priors overriding evidence** is the expensive one: it requires a judge that compares the final answer against the observations actually retrieved, which is a model-graded eval and costs real money to run. So the honest ranking for most domains is not by frequency but by **cost of the mistake divided by cost of detection**. In a regulated domain — banking, health, anything with a regulator — the third one dominates, because a confident answer that contradicts the retrieved document is precisely the failure that survives review and reaches a customer. That is also why so much of Week 6's guardrail material is about checking output *against sources* rather than checking output alone: it is the only one of the three you cannot catch with a counter.

APPENDIX — LOOP VOCABULARY, EXPLAINED

*One distinction carries this reading: **which tokens the model wrote, and which tokens the world wrote**. Everything below sits on one side of that line or the other — or names something you do to the text once it has accumulated.*

Thought

- **Plain.** A line of reasoning the model writes to itself before acting — “I need X, so I will look up Y.”
Technically. Free-form text in the trajectory; not sent to any tool. Its function is to condition the next action token. Removing thoughts (act-only prompting) degrades multi-hop performance sharply.

Action

- **Plain.** A structured instruction to do something — call a tool, with arguments.
Technically. Emitted as tokens matching a tool schema, parsed by your code and executed. In 2022 this was parsed from text; in 2026 the same thing is done through native tool-calling APIs that return a structured call. Same loop, better plumbing.

Observation

- **Plain.** What the world sends back, pasted into the conversation.
Technically. The only tokens the model did not generate — the sole channel for external correction, and the main driver of context growth. Its size is the single highest-leverage cost knob in an agent.

Trajectory

- **Plain.** The whole run written out: every thought, action and observation in order.
Technically. The agent's entire state — these systems are stateless between turns and carry context only as accumulated text. This is why context windows fill, why cost grows with the square of turn count, and why compaction exists.

Prompt caching

- **Plain.** Pay full price to process the front of the prompt once, then a fraction of it on every later turn that begins with exactly the same text.
Technically. The provider keeps the processed form of a prefix — system prompt, tool schemas, early turns — and reuses it while the cache is warm (minutes, not hours). Cached reads bill at a fraction of the normal input price and a cache write carries a small premium; the exact ratios are provider-specific and change, so check current pricing. It requires an **exact, unchanged prefix**: a timestamp in the system prompt, a reordered tool list or an edit to turn 1 invalidates everything after it. Caching discounts the repeated term in the $T^2/2$ arithmetic above — it does not remove it, and it does nothing at all for context-window pressure.

Compaction

- **Plain.** Replacing a long stretch of earlier turns with a short summary of them, so the run can keep going.
Technically. Triggered as the trajectory approaches a context threshold: a model summarises the run so far — decisions taken, facts established, sub-goals still open — and that summary replaces the raw turns, with the most recent few usually kept verbatim. It buys back window and truncates the quadratic term. It is **lossy by construction**: a detail the summariser drops at turn 12 cannot be recovered at turn 30, and that surfaces as an agent contradicting something it once knew. It also breaks the cached prefix, so a compaction event pays a fresh cache write. The alternative is not to compress context but to move it out of the window — write state to a file or scratchpad and read it back on demand. Which of the two you reach for is a Capsule 2 discussion.

Grounding

- **Plain.** Tying what the model says to something outside the model.
Technically. In ReAct, achieved by inserting real tool results into the reasoning stream. Reading 1's phrase for the same requirement is “ground truth from the environment at each step”.

Act-only prompting

- **Plain.** The ablation: let it act, but forbid it from writing out its plan.
Technically. The paper's control condition. Isolates the contribution of interleaved reasoning; performance drops most on tasks requiring several dependent steps.

Self-consistency

- **Plain.** Ask the same question several times and take the majority answer.
Technically. Sampling multiple reasoning paths and voting (Week 3, and Reading 1's "parallelisation — voting"). Combined with ReAct it gave the paper's strongest knowledge-task results, and it multiplies cost by the number of samples.

Step cap / termination condition

- **Plain.** A hard limit on how many turns the loop may run before it stops.
Technically. Not in the paper; universal in production. With a token-budget ceiling and action deduplication, it converts the repetition failure mode from an unbounded bill into a logged, recoverable stop.

SOURCE LINKS

Paper — arXiv abstract and PDF · arxiv.org/abs/2210.03629

Project page with example trajectories · react-lm.github.io/ (*the figures are the fastest way in*)

Companion · **Reading 1** (Anthropic) is this loop seen from the outside — when it is worth running at all; **Reading 3** (LeCun) argues that a loop reasoning in tokens is the wrong substrate for planning in the first place.

CAVEATS

1. This summary is not a substitute for the paper. Read at least one full trajectory in the original figures — the mechanism is far more obvious in the raw text than in any description of it, including this one.
2. The trajectory in "The mechanism" is illustrative and written for this course; it is not from the paper, and the bank figures in it are placeholders for the shape of the reasoning, not verified financials.
3. Benchmarks and models are from 2022. The +34 and +10 point figures are the paper's own, on ALFWorld and WebShop respectively, against that era's baselines; do not quote them as current.
4. This summary was generated and fact-checked using Anthropic models. The cost arithmetic and the three-layer fix framing are the instructor's. Flag anything that looks wrong.